Protocolo CAN (Controller Area Network): Fundamentos, Arquitectura de Trama y Aplicaciones

Contreras R Orlando[1,2*]

[3]Department of Electronic Systems, UAA, Av. Universidad 940, Aguascalientes, 20131, Aguascalientes, México.

Corresponding author(s). E-mail(s): {al348390,al464376}@edu.uaa.mx;

Abstract

El presente reporte estudia el protocolo de comunicación serial *Controller Area Network* (CAN), desarrollado originalmente por Bosch para reducir el cableado en sistemas automotrices. Se abordan sus fundamentos eléctricos y de capa física, la estructura de sus tramas, el mecanismo de arbitraje sin pérdida basado en *CSMA/CR*, el manejo y clasificación de errores, así como su evolución hacia CAN FD. Se discuten ventajas y desventajas frente a otros buses de campo, sus aplicaciones actuales en la industria automotriz e industrial, y se presenta una guía práctica de implementación del periférico bxCAN en un microcontrolador STM32.

Keywords: CAN Bus, ISO 11898, Bus de campo, Arbitraje, STM32, Comunicación serial, CAN FD.

1 Introducción

El protocolo **CAN (Controller Area Network)** es un estándar de comunicación serial multi-maestro diseñado para permitir que microcontroladores y dispositivos se comuniquen entre sí sin necesidad de una computadora central, dentro de un mismo sistema. A diferencia de protocolos punto a punto como SPI o UART, CAN fue concebido desde su origen como un bus multi-nodo: todos los dispositivos "escuchan" el mismo par de cables y deciden, mediante un mecanismo de arbitraje, quién tiene el derecho de transmitir en cada instante.

Su principal motivación histórica fue la reducción drástica del cableado en los vehículos, donde cada vez más unidades de control electrónico (ECUs) necesitaban intercambiar información (velocidad, temperatura, posición del acelerador, estado de

frenos, etc.). En lugar de tender un cable dedicado entre cada par de módulos, CAN permite que todos los nodos compartan un único bus físico de dos hilos.

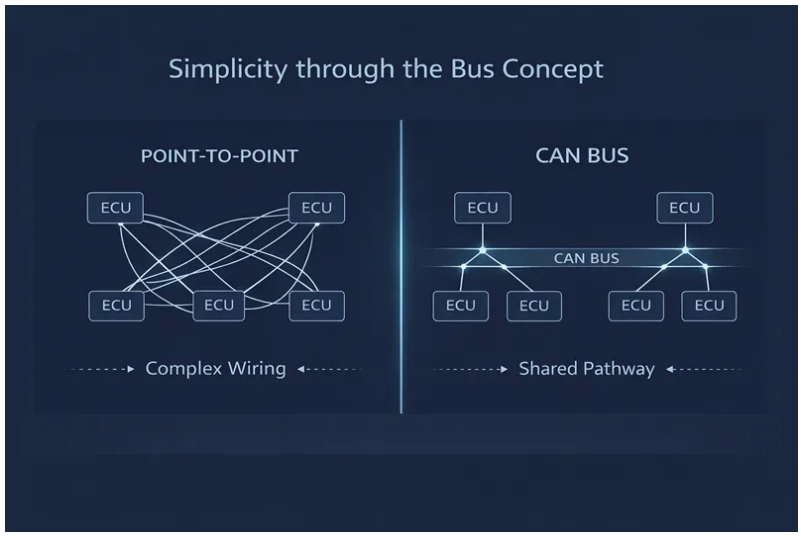


Fig. 1: Comparación entre cableado punto a punto tradicional y topología de bus CAN

2 Breve Historia

El desarrollo de CAN inició en 1983 dentro de Robert Bosch GmbH, liderado por Uwe Kiencke, como respuesta a la necesidad de un bus serial robusto que redujera el cableado automotriz. El protocolo se presentó públicamente en 1986 durante el congreso SAE en Detroit, y poco después Intel y Philips (NXP) lanzaron los primeros controladores CAN comerciales.

Mercedes-Benz fue el primer fabricante en integrar CAN en un vehículo de producción, el Clase S W140, en 1991; ese mismo año Bosch publicó la especificación CAN 2.0 (con las variantes 2.0A de identificador estándar de 11 bits y 2.0B de identificador extendido de 29 bits). En 1993 la ISO ratificó el protocolo como **ISO 11898**, consolidándolo como estándar internacional. A mediados de los años 90, CAN se incorporó a los sistemas de diagnóstico OBD-II, obligatorios en Estados Unidos desde 1996.

Con el crecimiento en la cantidad de datos a transmitir por los sistemas ADAS y de infoentretenimiento, Bosch introdujo en 2012 el protocolo **CAN FD** (*Flexible Data-Rate*), formalizado por ISO en 2015 dentro de ISO 11898-1:2015, permitiendo cargas útiles de hasta 64 bytes y una fase de datos a mayor velocidad.

3 Conceptos Básicos

3.1 Topología y Capa Física

CAN utiliza un bus diferencial de dos hilos, denominados **CAN_H** (*CAN High*) y **CAN_L** (*CAN Low*). La información no se codifica como un nivel absoluto de voltaje respecto a tierra, sino como la *diferencia de tensión* entre ambas líneas, lo que otorga a CAN una alta inmunidad al ruido electromagnético, característica esencial en el entorno automotriz.

Existen dos estados posibles en el bus:

- **Estado recesivo (bit 1):** Ambas líneas se encuentran aproximadamente al mismo potencial (típicamente 2.5V), por lo que la diferencia de voltaje es cercana a 0V.
- **Estado dominante (bit 0):** CAN_H se eleva (aprox. 3.5V) y CAN_L desciende (aprox. 1.5V), generando una diferencia de aproximadamente 2V.

El estado dominante siempre "gana" sobre el recesivo cuando dos o más nodos transmiten simultáneamente, lo cual es la base física que permite el arbitraje sin colisiones (sección 3.2).

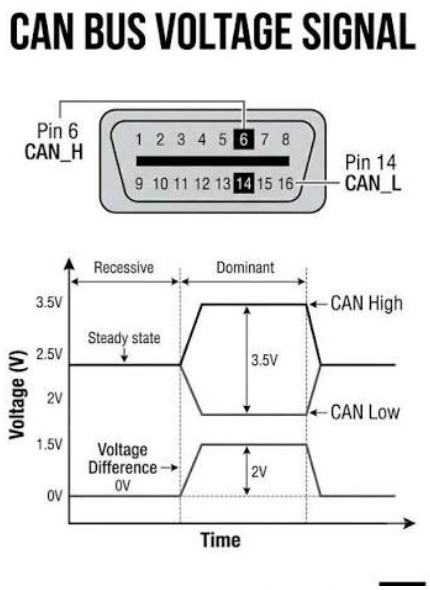


Fig. 2: Niveles de voltaje diferencial: estado dominante vs recesivo

Físicamente, cada nodo se conecta al bus mediante un **transceptor CAN** (por ejemplo, el TJA1050, TJA1057, MCP2551 o SN65HVD230), que traduce los niveles lógicos TTL del controlador CAN integrado en el microcontrolador (como el periférico bxCAN o FDCAN de la familia STM32) a los niveles diferenciales del bus físico. En ambos extremos del bus se colocan resistencias de terminación de 120 Ω para evitar reflexiones de señal, dando una resistencia diferencial total de 60 Ω.

3.2 Arbitraje y Acceso Múltiple: CSMA/CR

CAN es un bus **multi-maestro**: cualquier nodo puede iniciar una transmisión cuando detecta el bus libre. Para resolver el caso en que dos o más nodos transmiten al mismo tiempo, CAN emplea un mecanismo llamado **CSMA/CR** (*Carrier Sense Multiple Access with Collision Resolution*), también descrito como arbitraje bit a bit no destructivo.

El procedimiento es el siguiente: mientras se transmite el campo de identificador, cada nodo transmisor monitorea simultáneamente el estado real del bus. Si un nodo transmite un bit recesivo pero detecta que el bus está en estado dominante (porque otro nodo transmitió un 0 en esa misma posición), dicho nodo pierde el arbitraje, deja de transmitir inmediatamente y pasa a modo receptor, reintentando su transmisión cuando el bus vuelva a estar libre. El nodo cuyo identificador contiene el valor

numérico más bajo -es decir, más bits dominantes al inicio- gana el arbitraje y continúa transmitiendo sin interrupción.

Esta característica implica que **el identificador de un mensaje CAN determina directamente su prioridad**: entre menor sea el valor numérico del identificador, mayor es su prioridad en el bus. Además, este mecanismo asegura que el mensaje de mayor prioridad nunca se pierde ni se corrompe por la colisión, a diferencia de protocolos como Ethernet clásico (CSMA/CD), donde ambas tramas en colisión se descartan y deben retransmitirse.

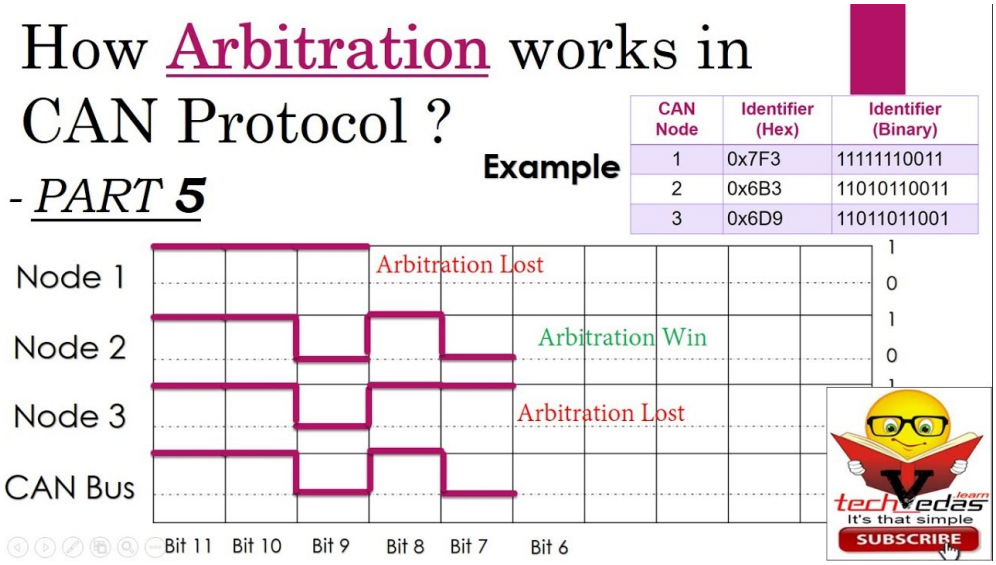


Fig. 3: Arbitraje bit a bit (CSMA/CR) entre tres nodos compitiendo por el bus

4 Estructura de la Trama CAN

CAN define cuatro tipos de tramas: **trama de datos** (transporta información), **trama remota** (solicita datos a otro nodo), **trama de error** (señaliza una violación del protocolo) y **trama de sobrecarga** (retrasa el siguiente envío cuando un nodo requiere más tiempo de procesamiento). La trama de datos es la más relevante para el uso cotidiano del bus y se describe a detalle a continuación.

4.1 Trama de Datos: Formato Estándar (CAN 2.0A)

El identificador estándar utiliza 11 bits. Los campos de la trama, en orden de transmisión, son:

4.2 Trama de Datos: Formato Extendido (CAN 2.0B)

El formato extendido amplía el identificador a 29 bits para admitir un mayor número de mensajes únicos, requerido por protocolos de capa superior como J1939. Se logra dividiendo el identificador en un identificador base de 11 bits y una extensión de 18 bits adicionales, separados por los bits **SRR** (*Substitute Remote Request*) e **IDE** (que en este caso se transmite recesivo para indicar formato extendido).

Es importante destacar que, gracias al bit IDE, un identificador estándar siempre tiene mayor prioridad que un identificador extendido cuando ambos compiten en el mismo

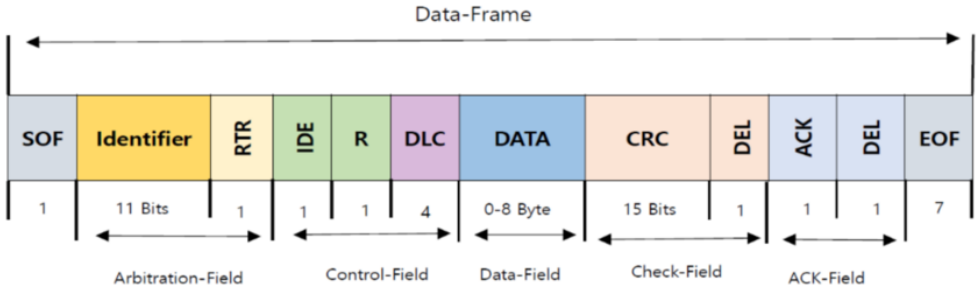


Fig. 4: Estructura completa de la trama de datos CAN estándar (CAN 2.0A)

Table 1: Campos de la trama de datos CAN estándar (11 bits)

Campo	**Tamaño**	**Descripción**
SOF (Start of Frame)	1 bit	Bit dominante que marca el inicio de la trama y sincroniza a todos los nodos.
Identificador	11 bits	Define la prioridad del mensaje y, típicamente, su significado/contenido.
RTR	1 bit	Dominante en trama de datos, recesivo en trama remota.
IDE	1 bit	Dominante: indica formato estándar (11 bits).
r0	1 bit	Bit reservado (dominante, reservado para uso futuro).
DLC (Data Length Code)	4 bits	Indica el número de bytes de datos (0 a 8).
Campo de datos	0–8 bytes	Carga útil del mensaje.
CRC	15 bits + 1 delimitador	Checksum para detección de errores de transmisión.
ACK	1 bit + 1 delimitador	Bit de reconocimiento: el transmisor lo envía recesivo, cualquier receptor que reciba correctamente lo fuerza a dominante.
EOF (End of Frame)	7 bits recesivos	Marca el final de la trama.

bit de arbitraje, ya que IDE se transmite dominante en tramas estándar y recesivo en tramas extendidas.

4.3 Bit Stuffing

Para garantizar suficientes transiciones de señal que permitan a los nodos mantener sincronía de reloj (recordemos que CAN es asíncrono, sin línea de reloj dedicada), el protocolo inserta un **bit de relleno (stuff bit)** de polaridad opuesta después de cada secuencia de 5 bits consecutivos del mismo valor, dentro de los campos comprendidos entre el SOF y el CRC. El receptor elimina automáticamente estos bits de relleno al reconstruir el mensaje original. Esta técnica también resulta clave para la detección de errores de relleno (*stuff error*), descrita en la sección 5.

4.4 Trama Remota, de Error y de Sobrecarga

- **Trama remota (Remote Frame):** Idéntica en estructura a la trama de datos, pero con el bit RTR recesivo y sin campo de datos. Se utiliza para solicitar a otro nodo que transmita la información asociada a un identificador específico.
- **Trama de error (Error Frame):** Se compone de una *bandera de error* (6 bits del mismo valor, violando la regla de bit stuffing intencionalmente) seguida de un *delimitador de error* (8 bits recesivos). Cualquier nodo que detecte una

de las condiciones de error descritas en 5 transmite esta trama para abortar la transmisión en curso.

- **Trama de sobrecarga (Overload Frame):** Estructuralmente similar a la trama de error, pero se genera cuando un nodo necesita más tiempo antes de procesar el siguiente mensaje, insertando un retraso adicional entre tramas.

5 Manejo y Clasificación de Errores

Uno de los aspectos más robustos de CAN es su sofisticado sistema de detección de errores, que le permite alcanzar una probabilidad de error no detectado extremadamente baja. El protocolo define cinco mecanismos de detección:

Table 2: Tipos de error detectados por el protocolo CAN

Tipo de error	**Descripción**
Bit Error	El transmisor compara cada bit enviado contra el nivel real del bus (fuera de la fase de arbitraje) y detecta una discrepancia.
Stuff Error	Se reciben 6 bits consecutivos del mismo valor donde debería existir un bit de relleno.
CRC Error	El CRC calculado por el receptor no coincide con el CRC recibido en la trama.
Form Error	Un campo de formato fijo (como el delimitador de CRC, ACK o el EOF) contiene un valor inválido.
ACK Error	El transmisor no detecta el bit de reconocimiento dominante, indicando que ningún nodo recibió la trama correctamente.

Cada nodo mantiene dos contadores internos, **TEC** (*Transmit Error Counter*) y **REC** (*Receive Error Counter*), que se incrementan al detectar errores y se decrementan con transmisiones/recepciones exitosas. En función del valor de estos contadores, un nodo transita entre tres estados:

- **Error Active:** Estado normal de operación; el nodo participa activamente en el bus y puede emitir tramas de error activas (dominantes).
- **Error Passive:** Se alcanza cuando TEC o REC superan 127; el nodo continúa comunicándose pero sus tramas de error pasan a ser pasivas (recesivas), para no interferir agresivamente con el bus.
- **Bus Off:** Se alcanza cuando TEC supera 255; el nodo se desconecta completamente del bus para evitar comprometer la comunicación del resto de la red, requiriendo una secuencia de recuperación definida por el estándar.

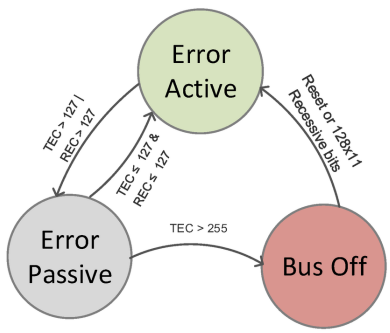


Fig. 5: Máquina de estados de confinamiento de fallas (fault confinement)

6 Capa Física: Variantes de CAN

El estándar ISO 11898 se subdivide en varias partes, de las cuales destacan dos variantes de capa física de uso común:

- **High-Speed CAN (ISO 11898-2):** Permite velocidades de hasta 1 Mbit/s, utilizado en aplicaciones sensibles al tiempo como el control del motor y frenos ABS. No es tolerante a fallas por sí mismo (una falla en una de las dos líneas puede detener la comunicación).
- **Low-Speed Fault-Tolerant CAN (ISO 11898-3):** Alcanza hasta 125 kbit/s, pero puede seguir operando en modo degradado utilizando una sola línea si la otra falla (por ejemplo, corto a tierra o a alimentación), a costa de menor velocidad. Se usa en sistemas de confort (elevadores de cristal, luces, asientos).

7 CAN FD (Flexible Data-Rate)

CAN FD, introducido por Bosch en 2012 y formalizado en ISO 11898-1:2015, extiende el protocolo CAN clásico manteniendo compatibilidad con el hardware existente en la fase de arbitraje. Sus mejoras principales son:

- **Carga útil ampliada:** El campo de datos crece de un máximo de 8 bytes a hasta 64 bytes por trama.
- **Velocidad de datos variable:** Una vez resuelto el arbitraje (que se mantiene a la velocidad nominal por compatibilidad), la fase de transmisión de datos puede conmutar a una velocidad más alta, ya que en ese punto solo un nodo transmite y no es necesario mantener los tiempos de propagación conservadores del arbitraje.
- **Mayor robustez del CRC:** Se emplea un polinomio CRC más robusto (17 o 21 bits, dependiendo de la longitud de datos) dado el mayor tamaño de carga útil.

Esta evolución responde al crecimiento en la cantidad de datos que deben intercambiar los sistemas ADAS, cámaras y unidades de infoentretenimiento modernas, sin necesidad de migrar a un bus completamente distinto.

8 Ventajas y Desventajas

Table 3: Ventajas y desventajas del protocolo CAN

Ventajas	**Desventajas**
Reduce drásticamente el cableado al usar un bus compartido de dos hilos.	Ancho de banda limitado (1 Mbit/s en CAN clásico), insuficiente para video o grandes flujos de datos.
Alta inmunidad al ruido gracias a la señalización diferencial.	Carga útil máxima de solo 8 bytes en CAN clásico, generando overhead relativo alto en mensajes pequeños.
Arbitraje sin pérdida: el mensaje de mayor prioridad nunca se corrompe por colisiones.	Sin direccionamiento de origen/destino nativo; cualquier nodo puede inyectar tramas con cualquier identificador (relevante en ciberseguridad).
Detección de errores robusta (CRC, bit stuffing, múltiples verificaciones), muy baja tasa de error no detectado.	Requiere transceptor dedicado y terminación adecuada; una falla de terminación degrada toda la red.
Estándar maduro, ampliamente soportado por microcontroladores (bxCAN, FDCAN) y bajo costo de implementación.	Escalabilidad limitada en número de nodos y longitud de bus a altas velocidades (dependencia del tiempo de propagación).
Determinismo en la prioridad de mensajes críticos gracias al identificador.	Sin cifrado ni autenticación en el protocolo base; toda la seguridad se delega a capas superiores.

9 Aplicaciones y Uso Actual

Aunque CAN nació en el sector automotriz, su robustez y bajo costo lo han llevado a diversificarse en múltiples industrias:

- **Automotriz:** Comunicación entre ECUs de motor, transmisión, frenos ABS/ESP, airbags, y diagnóstico OBD-II. Es la base de los buses de confort y de tren motriz en la gran mayoría de vehículos actuales.
- **Vehículos comerciales y agrícolas:** El estándar **SAE J1939**, construido sobre CAN de identificador extendido, define la comunicación entre ECUs en camiones, tractores y maquinaria pesada.
- **Automatización industrial:** El protocolo de capa de aplicación **CANopen** se utiliza extensamente en controladores de motores, sensores y PLCs industriales.
- **Electrónica marina:** El estándar **NMEA 2000**, basado en CAN, interconecta instrumentos de navegación, GPS y sensores en embarcaciones.
- **Aeroespacial y ferroviario:** Variantes de CAN se emplean en sistemas de control no críticos para la seguridad de vuelo, así como en trenes para puertas, climatización e iluminación.
- **Robótica y proyectos embebidos:** Muy común en drones (por ejemplo, buses CAN en controladoras de vuelo modernas para ESCs) y en robots móviles multi-nodo, dado el bajo costo de los transceptores y el soporte nativo en microcontroladores STM32, entre otros.

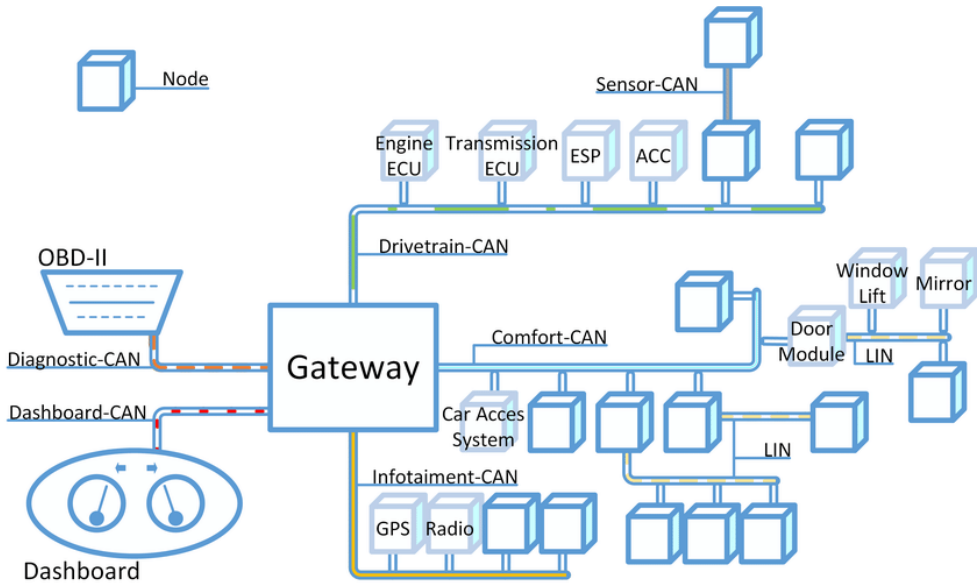


Fig. 6: Ejemplo de red vehicular con múltiples buses CAN interconectados mediante un gateway

10 Implementación Práctica en STM32

Los microcontroladores STM32 integran de forma nativa un periférico controlador CAN: **bxCAN** (*Basic Extended CAN*) en familias como F1, F4 y F103, o el más moderno **FDCAN** en familias G4, H7 y L5, este último con soporte completo para CAN FD. En ambos casos, es necesario un transceptor externo (por ejemplo, TJA1050 o SN65HVD230) entre los pines TX/RX del periférico y el bus físico CAN_H/CAN_L.

¿Qué hace el transceptor? El controlador CAN del STM32 trabaja con señales digitales de 3.3 V (TX y RX). El transceptor se encarga de:

- Convertir esas señales al nivel diferencial de CAN_H y CAN_L.
- Proteger el microcontrolador frente a perturbaciones del bus.
- Adaptar los niveles eléctricos según el estándar ISO 11898.

Sin él, no puedes conectar directamente el STM32 al bus CAN.

10.1 Conexión de Hardware

La conexión típica involucra los pines `CAN_TX` y `CAN_RX` del microcontrolador hacia el transceptor, y las salidas diferenciales del transceptor hacia el bus con su resistencia de terminación de 120 Ω en cada extremo físico del bus (no en cada nodo).

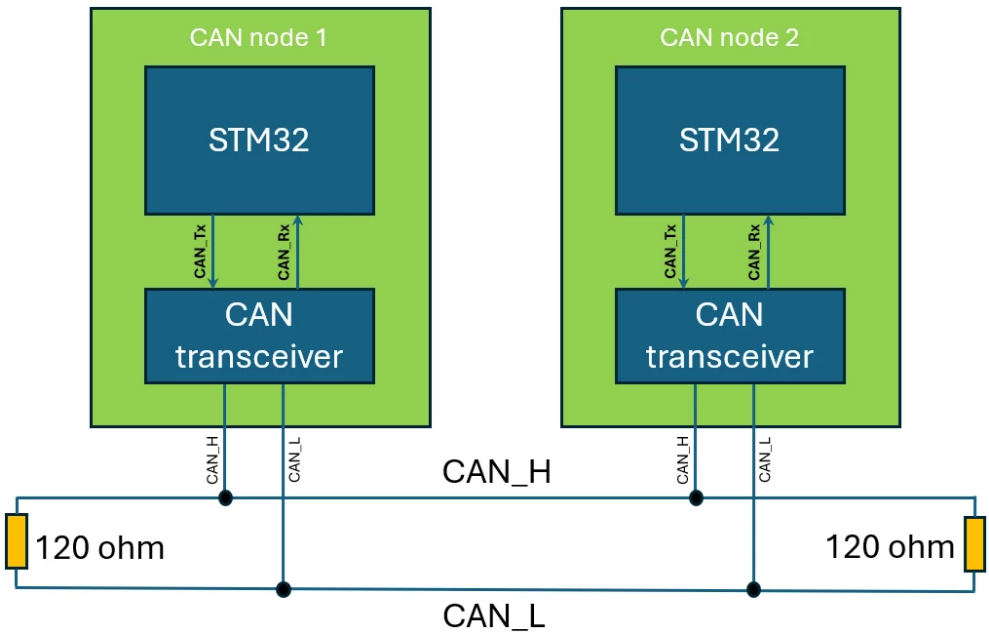


Fig. 7: Esquemático de conexión STM32 + transceptor CAN + bus terminado

Es recomendable utilizar el modulo **SN65HVD230** debido a que trabaja a 3.3 V, es compatible con el nivel lógico del STM32 y soporta velocidades de hasta 1 Mbit/s. Además, su consumo es bajo y tiene protección contra cortocircuitos y sobrevoltajes.

10.2 Configuración e Inicialización (bxCAN)

El siguiente ejemplo ilustra, a nivel conceptual, la configuración mínima del periférico bxCAN utilizando HAL para transmitir una trama estándar con identificador y dos bytes de datos.

Code: main.c -- Inicialización y envío bxCAN

```
CAN_HandleTypeDef hcan1;
CAN_TxHeaderTypeDef TxHeader;
uint8_t TxData[8];
uint32_t TxMailbox;

void CAN1_Config(void) {
    hcan1.Instance = CAN1;
    hcan1.Init.Prescaler = 4;              // Ajustar segun APB1 clock
    hcan1.Init.Mode = CAN_MODE_NORMAL;
    hcan1.Init.SyncJumpWidth = CAN_SJW_1TQ;
    hcan1.Init.TimeSeg1 = CAN_BS1_13TQ;
    hcan1.Init.TimeSeg2 = CAN_BS2_2TQ;      // Bit rate objetivo: 500 kbit/s
    hcan1.Init.TimeTriggeredMode = DISABLE;
    hcan1.Init.AutoBusOff = DISABLE;
    hcan1.Init.AutoWakeUp = DISABLE;
    hcan1.Init.AutoRetransmission = ENABLE;
    hcan1.Init.ReceiveFifoLocked = DISABLE;
    hcan1.Init.TransmitFifoPriority = DISABLE;

    HAL_CAN_Init(&hcan1);

    // Filtro: aceptar todos los identificadores (ejemplo simple)
    CAN_FilterTypeDef sFilterConfig;
    sFilterConfig.FilterBank = 0;
    sFilterConfig.FilterMode = CAN_FILTERMODE_IDMASK;
    sFilterConfig.FilterScale = CAN_FILTERSCALE_32BIT;
    sFilterConfig.FilterIdHigh = 0x0000;
    sFilterConfig.FilterIdLow = 0x0000;
    sFilterConfig.FilterMaskIdHigh = 0x0000;
    sFilterConfig.FilterMaskIdLow = 0x0000;
    sFilterConfig.FilterFIFOAssignment = CAN_FILTER_FIFO0;
    sFilterConfig.FilterActivation = ENABLE;
    HAL_CAN_ConfigFilter(&hcan1, &sFilterConfig);

    HAL_CAN_Start(&hcan1);
}

void CAN1_SendFrame(uint16_t id, uint8_t byte0, uint8_t byte1) {
    TxHeader.StdId = id;          // Identificador de 11 bits: define la prioridad
    TxHeader.ExtId = 0;
    TxHeader.IDE = CAN_ID_STD;    // Formato estandar
    TxHeader.RTR = CAN_RTR_DATA; // Trama de datos (no remota)
    TxHeader.DLC = 2;             // 2 bytes de payload
    TxHeader.TransmitGlobalTime = DISABLE;

    TxData[0] = byte0;
    TxData[1] = byte1;

    HAL_CAN_AddTxMessage(&hcan1, &TxHeader, TxData, &TxMailbox);
}
```

Puntos clave del ejemplo:

- El `Prescaler`, `TimeSeg1` y `TimeSeg2` determinan la velocidad de bit (*bit rate*) resultante a partir del reloj APB1; deben calcularse de modo que todos los nodos de la red compartan exactamente la misma velocidad nominal.
- El campo `StdId` corresponde directamente al identificador de arbitraje descrito en la sección 4: entre menor su valor, mayor prioridad tendrá la trama en el bus.

- `AutoRetransmission = ENABLE` delega al hardware bxCAN el reintento automático de una trama tras perder el arbitraje o detectar un error, replicando el comportamiento estándar del protocolo.

10.3 Recepción por Interrupción

Para recibir mensajes es recomendable habilitar la interrupción de FIFO 0 con nuevo mensaje pendiente (`CAN_IT_RX_FIFO0_MSG_PENDING`) y sobrescribir el callback `HAL_CAN_RxFifo0MsgPendingCallback`, leyendo la trama con `HAL_CAN_GetRxMessage` para evitar sondeo (*polling*) continuo del periférico y reducir el uso de CPU, de forma análoga a las mejoras de DMA/interrupciones discutidas para otros periféricos SPI.

11 Referencias y Recursos para Profundizar

A continuación se listan fuentes primarias y recursos prácticos para profundizar en lo revisado en este reporte, organizados por categoría.

11.1 Estándares y especificaciones oficiales

- Bosch, *CAN Specification Version 2.0* (documento original, formatos estándar y extendido): http://esd.cs.ucr.edu/webres/can20.pdf
- CAN in Automation (CiA), *History of CAN Technology* (resumen histórico y estado actual de ISO 11898/CAN FD/CAN XL): https://www.can-cia.org/can-knowledge/history-of-can-technology
- CAN in Automation (CiA), portal oficial de la organización (CANopen, perfiles de dispositivo, documentación): https://www.can-cia.org/
- Wikipedia, *CAN bus* (panorama general con referencias cruzadas a CAN FD y CAN XL): https://en.wikipedia.org/wiki/CAN_bus

11.2 Datasheets de transceptores (capa física)

- NXP, *TJA1050 – High Speed CAN Transceiver*: https://www.nxp.com/docs/en/data-sheet/TJA1050.pdf
- Texas Instruments, *SN65HVD230/231/232 – 3.3V CAN Bus Transceivers*: https://www.ti.com/lit/ds/symlink/sn65hvd230.pdf

11.3 Implementación en STM32 (bxCAN/HAL)

- ST Community, *Using CAN (bxCAN) in Normal mode with STM32 microcontrollers – Part 1* (teoría, hardware y configuración): https://community.st.com/t5/stm32-mcus/using-can-bxcan-in-normal-mode-with-stm32-microcontrollers-part/ta-p/800502
- ST Community, *Using CAN (bxCAN) in Normal mode with STM32 microcontrollers – Part 2* (ejemplo práctico completo): https://community.st.com/t5/stm32-mcus/using-can-bxcan-in-normal-mode-with-stm32-microcontrollers-part/ta-p/810853
- ST Community, *CAN (bxCAN) bit time configuration on STM32 MCUs* (cálculo de `CAN_BTR`, prescaler y segmentos de tiempo): https://community.st.com/t5/stm32-mcus/can-bxcan-bit-time-configuration-on-stm32-mcus/ta-p/689864
- ST Community, *Guide to CAN (bxCAN/CAN2.0) configuration in Loop back mode on STM32 MCUs* (útil para probar sin un segundo nodo físico): https:

//community.st.com/stm32-mcus-60/guide-to-can-bxcan-can2-0-configuration-in-loop-back-mode-on-stm32-mcus-146788

- ControllersTech, *STM32 CAN Bus Communication in Normal Mode: CubeMX Setup & HAL Code* (tutorial con proyecto descargable, dos nodos STM32 + MCP2551): https://controllerstech.com/can-protocol-in-stm32/

11.4 Herramientas y librerías de software

- *python-can*, librería de Python para CAN sobre SocketCAN, PCAN, Vector, Kvaser, etc.: https://python-can.readthedocs.io/
- *python-can*, repositorio en GitHub: https://github.com/hardbyte/python-can
- Documentación de la interfaz SocketCAN dentro de python-can (creación de interfaces virtuales `vcan`, uso de `cansend`/`candump`): https://python-can.readthedocs.io/en/stable/interfaces/socketcan.html

12 Conclusión

El protocolo CAN se ha consolidado durante casi cuatro décadas como el bus de comunicación de referencia en sistemas embebidos distribuidos, gracias a un diseño elegante que resuelve de forma determinista el acceso múltiple al medio mediante arbitraje bit a bit, y a un sistema de detección de errores excepcionalmente robusto para el bajo costo de su capa física. Si bien su ancho de banda y tamaño de carga útil resultan limitados frente a las demandas actuales de los vehículos autónomos y conectados, su evolución hacia CAN FD -y en desarrollo, CAN XL- demuestra la vigencia del enfoque original. Para el trabajo práctico con microcontroladores STM32, el periférico bxCAN/FDCAN ofrece una vía directa y bien soportada por HAL para integrar nodos CAN en proyectos de instrumentación, robótica o redes de sensores distribuidos.